

POLITECNICO
MILANO 1863

COMPUTER SCIENCE AND ENGINEERING
SOFTWARE ENGINEERING II
2025 - 2026

I&TD

Implementation and Test Deriverable

Software:

Best Bike Paths (BBP)

Link for Web-app version:
https://zheng-yang-dai.vercel.app

Source Code(GitHub repository link):
https://github.com/SleepyEveryD/ZhengYangDai

Team:
Jiayi Zheng
Jiaxin Yang
Run Jie Simone Dai

Contents

1. Introduction and scope..... 3
2. The requirements/functions implemented in the software 4
3. Adopted development frameworks.....7
   - 3.1. Adopted Programming..... 7
   - 3.2. Adopted Middleware.....8
   - 3.3. External Services..... 9
   - 3.4. API not included..... 9
   - 3.5. Implementation Scope vs Design Assumption.....9
4. The structure of the source code..... 11
5. Information on how testing are performed.....12
   - 5.1. Testing Strategy and Procedure..... 12
   - 5.2. System Testing.....12
6. The installation instructions.....14
   - 6.1. Prerequisites..... 15
   - 6.2. Environment Configuration.....15
   - 6.3. Installation Procedure..... 15
   - 6.4. Additional Notes..... 15
7. Effort Spent..... 16
8. References..... 17

1| Introduction and Scope

This document provides the Installation and Technical Description (ITD) of the *Best Bike Paths* (BBP) application.
Its purpose is to describe the functionalities actually implemented in the system, the adopted technologies, the structure of the source code, the testing activities performed, and the installation procedure.

This document is intended for instructors and other student groups who need to install, run, and understand the implemented software.
Detailed design choices are described in the Design Document (DD) and are referenced where appropriate.

2| The Requirements/Functions that are actually implemented in the Software

This section describes the requirements and functionalities that have been actually implemented in the software.
The selection of implemented features was driven by the goal of delivering a functional and coherent cycling activity tracking application within the available development time.

For clarity, the functionalities described in this section represent the most significant and representative implemented features of the software, rather than an exhaustive list of all minor functionalities and system behaviors.
Several low-level technical functionalities and auxiliary behaviors, such as internal validations, error handling mechanisms, and user interface logic, are not explicitly listed, as they support the core features described below.

2.1 User Registration and Authentication

The system supports user registration and authentication through an email-based login mechanism.
Each user is uniquely identified and authenticated before accessing the core functionalities of the application.

This functionality was implemented to ensure that all cycling activities and related data are associated with a specific user.
Authentication is required in order to prevent unauthorized access to personal data and to enable persistent storage of user-specific ride information.

Advanced authentication features such as social login or multi-factor authentication were not included, as they were not essential for the core objectives of the project and would have increased implementation complexity.

2.2 Trip Recording

Registered users can start and stop a cycling trip using the mobile application.
When a trip recording session is initiated, the system verifies that location permissions and GPS services are available on the device.
If the required permissions are granted, the system begins acquiring GPS data during the ride.

During trip recording, the system continuously collects location updates and computes basic trip statistics, including total duration, traveled distance, and average speed.
These statistics are initialized at the beginning of the recording process and updated throughout the cycling activity.

This functionality represents the core feature of the application, as it enables users to track and analyze their cycling performance.
The focus was placed on reliability and correctness of collected data rather than on advanced real-time analytics.

2.3 Trip Data Storage

At the end of a recording session, the collected trip data is stored in the backend database and associated with the authenticated user.
Stored information includes trip duration, distance, average speed, and the sequence of recorded GPS coordinates.

Persistent storage of trip data allows users to access their past rides across multiple application sessions and devices.
This functionality was implemented to support long-term usage of the application and to enable future extensions, such as statistical summaries or comparisons between rides.

2.4 Trip History Visualization

The application allows users to view a list of their previously recorded trips.
For each trip, summary information such as date, duration, and distance is displayed.

This functionality was included to provide users with a clear overview of their cycling activity history and to improve the usability of the application.
Only essential summary information is presented in order to keep the user interface simple and easy to navigate.

Detailed map-based visualization of routes was not fully implemented, as it would require additional graphical components and more advanced data processing.

2.5 Manual Post-Trip Information Input

After completing a cycling trip, users can manually input additional information related to the ride.
In particular, the system allows users to provide qualitative feedback about the road conditions encountered during the trip.

This functionality is only available after a trip has been successfully recorded, ensuring that manual information is always associated with an existing ride.
The name of the road or route is not required; users can simply describe the conditions experienced.

The motivation for including this feature is to complement automatically collected data with user-provided context, which cannot be reliably obtained through sensors alone.

2.6 Excluded or Partially Implemented Requirements

Some functionalities initially considered during the requirements analysis phase were intentionally excluded or only partially implemented.

In particular, automatic road condition detection was not implemented.
The system does not automatically infer road conditions based on sensor data or GPS patterns. Instead, road condition information can only be provided manually by the user after completing a trip.

Additionally, the system does not include automatic activity detection.
The application does not automatically determine whether the user is cycling, nor does it start or stop trip recording based on movement patterns.
Users are required to manually initiate and terminate each cycling session.

These features were excluded due to their higher technical complexity and the need for advanced data processing, sensor fusion, and machine learning techniques, which were beyond the scope of this project.

By explicitly requiring user interaction for trip recording and road condition input, the system remains simpler, more predictable, and easier to test, while still fulfilling the core objectives of the application.

3| Adopted Development Frameworks

This section describes the development frameworks, programming languages, middleware, and APIs adopted for the implementation of the Best Bike Paths (BBP) prototype.
The choices presented here are consistent with the architectural design described in the Design Document (DD), while taking into account the prototype nature of the implementation.

Where appropriate, this section refers to the DD and highlights simplifications or deviations introduced during implementation.

3.1 Adopted Programming

JavaScript

JavaScript, with the adoption of TypeScript for improved type safety and maintainability, has been used for the implementation of the web-based frontend of the Best Bike Paths (BBP) prototype, in accordance with the web-oriented architecture described in the Design Document.

Modern JavaScript frameworks, such as React, provide built-in mechanisms that help reduce common client-side vulnerabilities. By enforcing a structured, component-based development model, these frameworks improve the robustness and maintainability of the frontend layer.
However, client-side technologies alone cannot guarantee the protection of sensitive data. In the BBP system, all security-critical aspects, such as authentication, authorization, and data validation, are enforced on the server side, consistently with the architectural decisions defined in the Design Document. The frontend is therefore treated as an untrusted component and is responsible solely for presentation and user interaction.

Another key advantage of JavaScript is its relatively low runtime and deployment overhead. JavaScript-based applications can be executed directly within standard web browsers on the client side or through lightweight runtimes such as Node.js on the server side, without requiring complex software installations or specialized hardware infrastructure. This characteristic makes JavaScript particularly suitable for web-based and prototype-level applications, where ease of deployment, portability, and fast startup times are essential. In the context of the BBP system, this choice simplifies installation and execution for evaluators and other student groups, while still providing adequate performance for the implemented functionalities.

Node.js

Node.js, through the adoption of the NestJS framework, has been used to support backend-related logic and the execution of server-side components during the development of the Best Bike Paths (BBP) prototype. This choice is consistent with the client–server and REST-based architecture described in the Design Document, where backend services expose RESTful APIs consumed by the web client.

Node.js provides an event-driven and non-blocking execution model that is well suited for handling multiple concurrent HTTP requests, which aligns with the interaction patterns of the BBP system, such as trip recording, bike path reporting, and route search. For a prototype-level implementation, this model offers sufficient performance while keeping the backend lightweight and easy to deploy.

A limitation of Node.js is that it may be less suitable for CPU-intensive computations due to its single-threaded execution model. However, the BBP prototype mainly performs I/O-bound operations, such as handling HTTP requests, processing user input, and interacting with data storage, which makes Node.js an appropriate and effective choice within the scope of the implemented system.

3.2 Adopted Middleware

RESTful HTTP APIs

The interaction between frontend and backend components is implemented through RESTful APIs over HTTP, in accordance with the client–server and RESTful communication patterns described in the DD (Section 2.6).

This choice ensures a clear separation between presentation logic and application logic, in line with the layered and client–server architectural principles described in the Design Document. RESTful communication also simplifies testing and debugging activities, as standard tools can be used to inspect requests and responses, and it allows a straightforward mapping between implemented endpoints and the API interfaces defined in the DD.

However, RESTful communication is inherently stateless, which requires explicit management of session-related and authentication information at the application level. In addition, HTTP-based communication is generally less efficient than binary protocols for high-frequency data exchange. This limitation does not represent a significant issue for the BBP prototype, given its scope and expected workload.

Express.js

The backend prototype adopts Express.js as middleware to manage HTTP routing and request handling. Express.js provides a lightweight and flexible framework that allows the implementation of backend endpoints with minimal configuration overhead. Its routing mechanisms enable a clear correspondence between implemented services and the conceptual components described in the DD. At the same time, Express.js requires developers to explicitly address concerns such as security, scalability, and fault tolerance, which are only partially implemented in the prototype due to scope constraints.

3.3 External Services Integration

The integration with Google Maps services is realized at the presentation layer of the prototype, within the web client responsible for route search and visualization. When a user performs an origin–destination query or selects a bike path for visualization, the frontend invokes Google Maps APIs to render the corresponding route geometry on an interactive map. This behavior directly implements the Map Service interactions described in the Design

Document and validates the feasibility of delegating map rendering and geolocation support to an external mapping provider.

Meteorological data services are integrated within the backend application logic responsible for trip management. After a trip is completed or stored, the system issues a request to an external weather API using the temporal and geographic attributes of the recorded trip. The retrieved weather information is then associated with the corresponding trip data and made available for visualization, in accordance with the responsibilities and interfaces defined in the Design Document.

3.4 API not included in DD

No additional external APIs beyond those already described in the Design Document have been introduced in the implementation. All external services integrated in the prototype, including mapping, geolocation, and meteorological services, were already modeled in the DD at an architectural level. The implementation therefore remains fully consistent with the original design choices, without introducing new external dependencies.

3.5 Implementation Scope vs Design Assumptions

Prototype Deployment Simplification

The Design Document describes a deployment architecture that includes replicated application servers, replicated databases, and load balancing mechanisms, in order to address scalability, availability, and fault tolerance requirements. These elements represent the target deployment architecture of the BBP system and are intended to support future production-level usage.

In the prototype implementation described in this deliverable, these deployment aspects are intentionally simplified. The system is deployed using a single application server and a single database instance, without replication or load balancing. This choice reflects the prototype-oriented scope of the ITD and allows us to focus on validating core functionalities, architectural interactions, and system behavior.

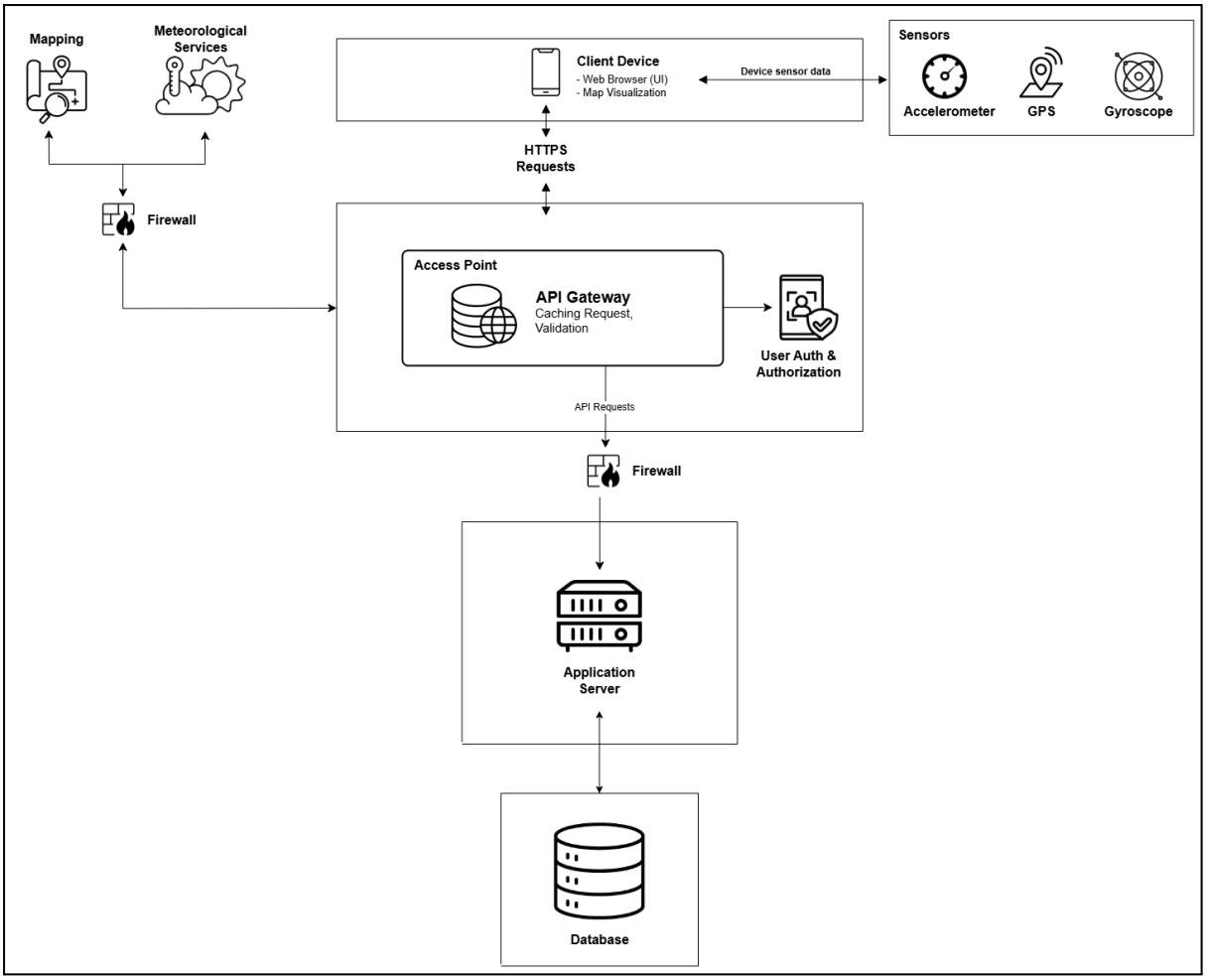


Figure: Architectural Design according to our Implementation

4| Structure of the Source Code

The source code of the software is organized using a clear separation between frontend and backend components, in order to improve readability, maintainability, and overall system scalability.

The overall project structure is as follows:

```
/Project
 ├── frontend/
 │   ├── src/
 │   │   ├── adapters/
 │   │   ├── auth/
 │   │   ├── components/
 │   │   ├── hooks/
 │   │   ├── lib/
 │   │   ├── services/
 │   │   ├── auth/
 │   │   ├── types/
 │   │   ├──app.tsx
 │   │   ├── main.tsx
 │   │   ├── routes.tsx
 │   └── package.json
 │   └──index.html
 │
 ├── backend/
 │         |── Prisma
 │          ├──src/
 │                ├── auth/
 │                ├── map/
 │                ├── prisma/
 │                ├── profile/
 │                ├── ride/
 │                ├── report/
 │                └── app.controller.ts
 │                 └── app.module.ts
 │                 └── app.service.ts
 └── package.json
```

The frontend directory contains the client-side application, which is responsible for user interaction, trip recording control, and data visualization.
It is organized into reusable components, pages corresponding to application views, and service modules used for communication with the backend APIs.

The backend directory contains the server-side logic of the application.
It includes controllers that expose RESTful endpoints, service layers that implement business logic, and authentication and authorization mechanisms that protect user-specific resources.

The database directory contains the database schema definition and related scripts used to support data persistence.

This modular organization helps keep responsibilities clearly separated and simplifies debugging, maintenance, and future extensions.

5| Information on how testing are performed

This section describes the testing activities performed during the development of the software.
The testing approach partially refers to the strategy described in the Design Document (DD) and extends it based on the actual implementation.

5.1 Testing Strategy and Procedure

Due to the limited scope and timeframe of the project, testing was mainly conducted through manual execution and runtime verification.

The testing strategy focused on:

- Verifying that the system can be successfully built and started without runtime errors.
- Executing the main user interaction flows and observing system behavior.
- Using console output on both frontend and backend sides to validate execution paths, state changes, and API interactions.
- Console logging was intentionally adopted as a lightweight testing and debugging mechanism to trace internal behavior and confirm that the system reacts correctly to user actions.

The overall testing procedure was as follows:

1. Start the backend services and the frontend application.
2. Perform specific user actions within the application.
3. Observe console output on the client and server sides.
4. Compare the observed behavior with the expected system behavior.

Particular attention was paid to error handling and abnormal situations, such as missing permissions or invalid user operations.

5.2 System Testing

System testing focused on validating the integration between frontend and backend components and ensuring that the system satisfies its main functional requirements.

The following representative system test cases were considered:

- Application Startup Test
  Input: start backend and frontend services
  Expected Result: the system starts successfully without runtime errors
  Outcome: passed

- User Authentication Flow Test
  Input: user login request
  Expected Result: authentication requests reach the backend and valid responses are observed in the console
  Outcome: passed

- Trip Recording Execution Test
  Input: manual start and stop of a cycling trip
  Expected Result: trip state changes and related data are correctly logged in the console
  Outcome: passed
- Data Persistence Test
  Input: completion of a trip recording
  Expected Result: console logs confirm successful data storage
  Outcome: passed

These tests indicate that the core functionalities of the system operate correctly and that the main integration points between components behave as expected.

6| The Installation Instructions

This section describes the steps required to install and run the software locally.

6.1 Prerequisites

To run the software locally, the following prerequisites are required:

- Node.js (version 24.11.0 or compatible)
- npm (version 11.6.1 or compatible)
- Prisma ORM (version 5)
- Stable internet connection

Both the frontend and backend components rely on npm for dependency installation and package management.

6.2 Environment Configuration

The backend relies on environment variables for database access and authentication configuration.

For testing purposes, a dedicated `.env` configuration file is required.
Due to security reasons, this file is not included in the public repository.

To obtain a test version of the `.env` file, please contact the project authors.

6.3 Installation Procedure

1. Clone the project repository from the provided source code link.
2. Navigate to the backend directory and install dependencies:
    a. npm install
    b. npm install prisma@5
    c. npx prisma generate
    d. npx prisma migrate deploy
3. Start the backend service:npm run start:dev
4. Navigate to the frontend directory and install dependencies:
    a. npm install
5. Start the frontend application:
    a. npm run dev

6.4 Additional Notes

- Both frontend and backend services must be running simultaneously for the system to function correctly.
- Prisma is required only for backend execution and database interaction.
- If the `.env` file is missing or incorrectly configured, the backend service will not start.
- GPS and location permissions must be enabled to test trip recording functionalities.

Network Considerations:
When running the software locally, it is recommended to avoid using university networks or public Wi-Fi connections.
In some network environments, outbound connections required by the database service and Prisma may be restricted, which can prevent the backend from connecting correctly.
For reliable execution, a private or unrestricted network connection is recommended.

7| Effort Spent

The following table summarizes the approximate number of hours each group member devoted to the different sections of the document. The reported distribution is indicative, as the preparation of each section required collaboration and joint discussion among all group members.

	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5	Chapter 6	Total
Run Jie Simone Dai	0	0	0	1	2	2	5
Jiaxin Yang	0.5	2.5	0.5	0	0.5	0	4
Jiayi Zheng	0	0.5	3.5	0	0	0	4

8| References

1. OpenAI: *ChatGPT*.
   Large Language Model used as a support tool for improving writing clarity, revising grammar. The tool was used to generate draft formulations, which were subsequently reviewed, validated, and adapted by the authors to ensure correctness and consistency with the Design Document and the implemented code.

2. Figma: *Figma Design Tool*.
   Collaborative design platform used for creating user interface mockups and diagrams during the design and documentation phases. The produced artifacts were manually reviewed and aligned with the system requirements and architectural decisions described in the Design Document.